

Implementing Traceability Repositories as Graph Databases for Software Quality Improvement

Randa Elamin

Faculty of Mathematical Sciences
University of Khartoum
Khartoum, Sudan
elamin.randa@gmail.com

Rasha Osman

Faculty of Mathematical Sciences
University of Khartoum
Khartoum, Sudan
rosman@ieee.org

Abstract—Traceability identifies dependencies between software artifacts facilitating the impact analysis of modifications to requirements, design and code. There is limited application of traceability in industry due to the complexity of traceability models and lack of tools. In this paper, we present simplified rules to define trace link types. To store and represent trace links, we implement a traceability repository as a native graph database. This is in contrast to other approaches that use structured files for storage or traceability matrices for representation. In addition, we present a methodology to apply our proposed rules to create trace links using three datasets. We demonstrate the advantage of the graph traceability repository over current representation and storage methods in visualizing traceability links, facilitating the derivation of new trace links and in query response times.

Keywords—traceability, trace type links, traceability metamodels, graph traceability repositories.

I. INTRODUCTION

Traceability is the ability to establish a relationship (trace link) between two or more software artifacts within the software development lifecycle [1]. This relationship can link the artifact to its source (backward direction) or to its dependent artifacts (forward direction). A traceability repository stores artifacts and trace links and facilitates the querying, maintenance and analysis of traceability information [2]. An important benefit of traceability is change management and change impact analysis. The ability to retrieve all artifact dependencies and determine the effect of a modification across the different development phases contributes to developing high quality and reliable software. In addition, traceability assists in estimating the required cost, time and resources to propagate modifications of software artifacts [3-5].

Traceability information models are used to represent software artifacts and their trace links. Fig. 1 details a traceability information model for requirement, test case and source code artifacts. Edges (trace links) represent dependencies between artifacts and edge labels (trace link types) define the meanings of the relationships. Trace link types are appended to trace links in order to define the semantics of the relationship between artifacts [6-8]. To define trace links and their semantics, current approaches employ the concept of a traceability metamodel which is a user-defined trace reference model [4, 9-12]. In this work, we extend the concept of user-defined metamodels in [12] by including trace link type rules to define the relationships between different artifacts within a software project. Our metamodels have simpler structures and formats compared to [12].

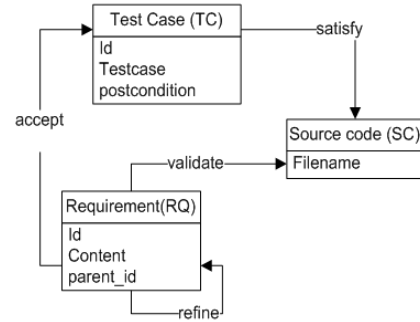


Fig. 1. Traceability information model for software artifacts.

Automated trace link generation and the derivation of new trace links minimize the impact of applying traceability within the software development process. There are limited approaches that address automatic generation of trace links. Most are based on parsing textual descriptions of software artifacts [13], similarity searching of terms [14, 15], or deriving new trace links based on the manual generation of the trace links [16, 17]. These approaches are limited by the complexity of the generation and derivation rules which restricts their applicability in industry-level projects.

In previous work [18], we proposed a methodology for implementing semi-automated trace link generation. To achieve such a goal our method should be simple, accurate and easily integrated within the software development process. We believe that the main limitation in previous traceability approaches and tools is the data structures for (1) the representation of traceability information and (2) the storage of such information. Most approaches represent traceability information as lists, cross reference tables and traceability matrices. These representations are suitable for one-to-one and one-to-many trace links but are limited for higher dimension relations [6, 19, 20]. Furthermore, most traceability approaches do not implement a traceability repository, which limit the ability to analyze trace information.

In this paper, we present a methodology to facilitate the creation of artifacts and their dependencies by applying simple patterns to define trace links and trace type rules. We implement the traceability repository as a native graph database given that the graph data structure is more suited for linked relationships [21]. Using three datasets, we show that the graph traceability repository natively (1) provides visible and readable trace representation [20, 22], (2) facilitates the discovery of derived links, (3) represents multi-dimensional trace

relations, and (4) executes the above features with less code using a special-purpose graph query language.

The main contributions of this paper are as follows.

- We present a simple user-defined traceability metamodel consisting of XML patterns for defining artifacts, trace links and trace type rules. Our trace type rules define the semantics of trace links and are simpler than other methods in the literature.
- We implement a traceability repository to represent artifacts, trace links and trace link types as an open source graph database (Neo4j). We show the benefit of this approach in comparison to commonly implemented representation and storage methods.
- We show that the graph traceability repository outperforms a relational repository in terms of query response times.
- We present a generic methodology that combines our metamodel patterns with the graph traceability repository and demonstrate its applicability to software artifact datasets.

This paper is organized as follows. Section II describes our methodology and its components. A comparison between our graph traceability repository and currently applied techniques is in Section III. Conclusions and future work are presented in Section IV.

II. METHODOLOGY

The main motivation of our method is to facilitate the implementation of traceability within lifecycle tools [18]. To achieve this, we demonstrate the application of our proposed metamodels and graph traceability repository through manual generation of artifacts and trace links. Manual generation corresponds to developers inserting artifact trace information within a lifecycle tool. In addition, we show the direct generation of traceability information from a predefined dataset and traceability model. The main components of our methodology are: the metamodel, the artifact and trace links repositories, the dataset preparation stage and the automated processes. Fig. 5 illustrates the components of the methodology and their interactions. The next sections provide the details of each component.

A. The MetaModel

The metamodel consists of three patterns: the *artifact model pattern*, the *trace link pattern* and the *trace type rule pattern*. We define the properties of each pattern within an XML file. A pattern instantiation is the mapping of the traceability information model to the artifact, trace link and trace type patterns. This is detailed below.

The artifact model pattern defines the traceability model artifacts and lists the collection of properties for each artifact using an XML format. Fig. 2 details the structure of the artifact model pattern and illustrates an instantiation of an artifact model instance from an artifact model pattern for the traceability information model in Fig. 1. The artifact model instance shows the definition of two artifacts: *requirement* and *test case* with their

properties (the *source code* artifact is not shown). From the artifact model instance, each artifact can then be defined, e.g., the requirement artifact is instantiated from the artifact model instance. The artifact model instance (XML) is utilized in the dataset preparation stage to create separate instances for each artifact defined in the dataset traceability information model.

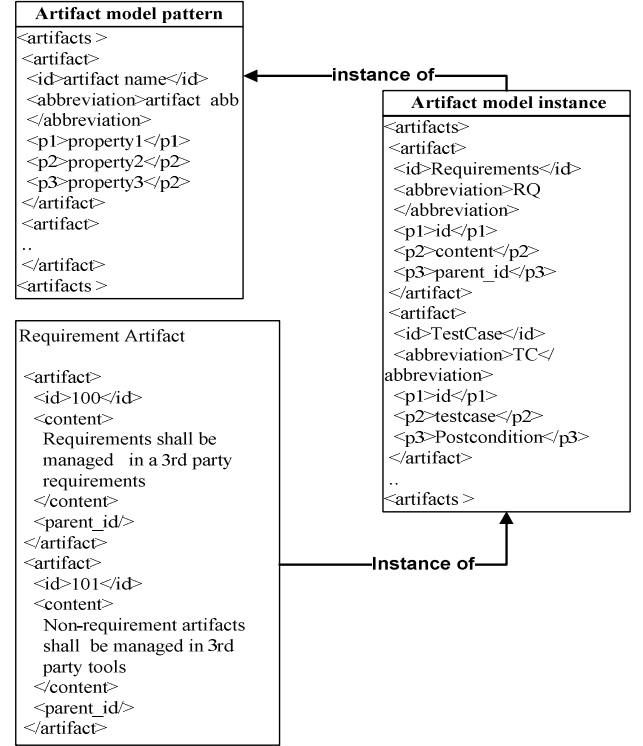


Fig. 2. An example of the generation of artifact model instances and artifact instances from artifact model patterns.

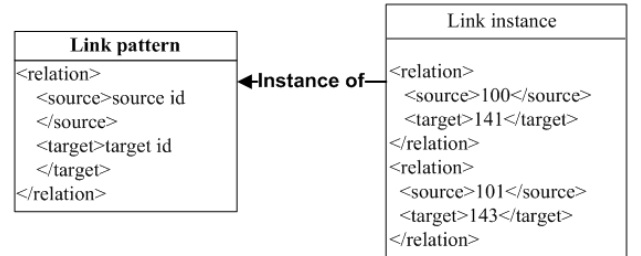


Fig. 3. An example of the generation of trace link instances from trace link patterns.

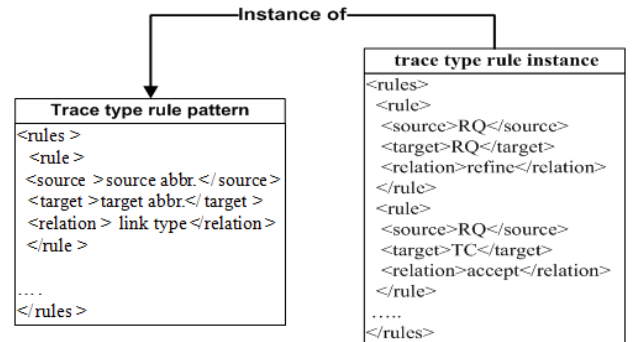


Fig. 4. An example of the generation of trace type rule instances from trace type rule patterns.

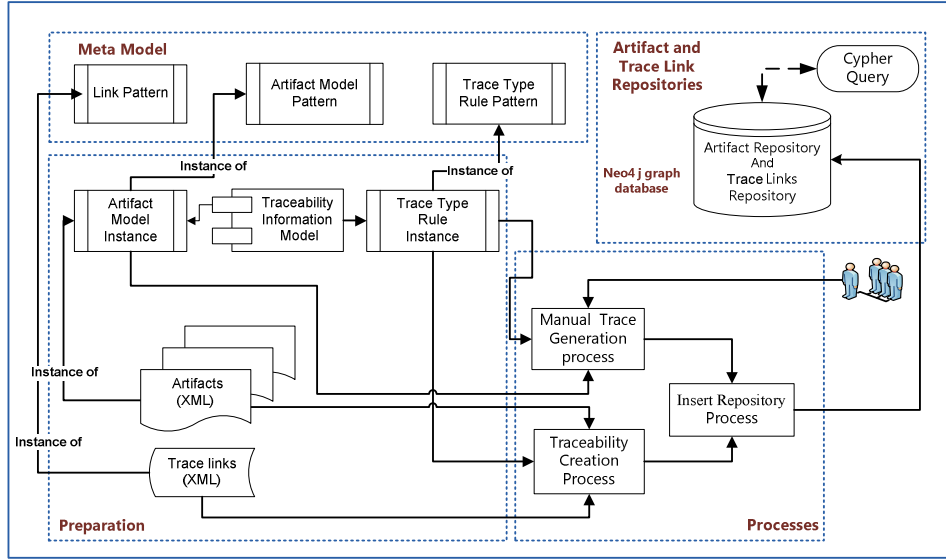


Fig. 5. Traceability information generation and storage methodology.

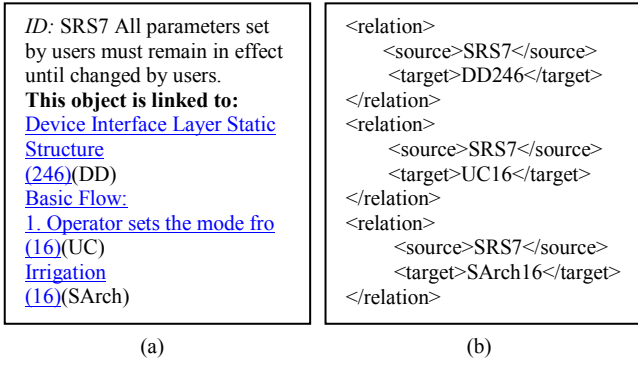


Fig. 6. Example of (a) dataset trace links (adapted from [23]) and (b) a dataset trace link instance.

The trace link pattern defines how to describe trace links between artifacts. From Fig. 3, the trace link pattern defines each relation between artifacts based on the *source artifact id* and the *target artifact id* specified in the artifact instances. Fig. 3 shows the trace link pattern and the corresponding trace link instance for the traceability model of Fig. 1. A trace link pattern is usually instantiated once for each dataset as it will represent all trace links within the dataset traceability model.

The trace type rule pattern defines the naming rules for all trace links defined within an instantiation of the trace link pattern. The structure of the trace type rule pattern is shown in Fig. 4, in which each relation between a source artifact instance and a target artifact instance is given a name. For example, Fig. 4 illustrates the trace type rule instance: if the *requirement* (RQ) artifact is related to another *requirement* (RQ) artifact then the trace link type will be *refine*. This trace link rule instance is instantiated during the trace links creation process as we discuss below.

The metamodel patterns are simple XML files that can be easily incorporated within an implementation of our methodology. All metamodel patterns are generic thus developers can define patterns that are specific to software applications and domains. In addition, the trace link rule pattern presents a generic user-defined rule for trace link

naming. Thus the semantics of trace links can be application specific. Furthermore, the use of XML allows for multi-lingual patterns and instances which provide for semantically rich traceability information.

B. Artifact and trace links repositories

An essential component for the automatic derivation and generation of trace links is combining the artifacts with their trace links in one repository. Implementing a traceability repository with full querying capabilities simplifies trace data manipulation and analysis, and provides complete and detailed trace information. To achieve complete representation of artifacts and trace links we exploit the capabilities of graph databases in naturally representing graph and linked data.

In a graph database, the artifact instances are represented as nodes, labels represent artifact types and node properties represent artifact details. Trace links are represented as relations in which the source artifact is the start node and the target artifact is the end node. The trace link type is represented as a relation type. We implemented our traceability repository using the Neo4j open source graph database. All artifacts instances are inserted into the graph database through process execution as described in the next Section. The graph traceability repository is accessible for searching and analyzing using the Cypher graph query language.

C. Dataset Preparation

Dataset artifacts exist in different formats, e.g., the datasets used in this work have XML and HTML formats. The dataset preparation stage is required given the existence of a dataset and its trace links. A transformation procedure is thus required to convert the dataset artifacts and trace links to formats suitable for our methodology. Our methodology implicitly builds the dataset traceability information model and accordingly generates the metamodel patterns. Based on the artifact model instances, for each dataset artifact within the traceability information model, an XML artifact instance is generated. Currently, the data preparation stage is

manual. The dataset trace links are represented by an XML trace link instance. The trace link rule instance preserves the semantics of the relations between database artifacts. Fig. 6(a) shows an example of trace links expressed as hyperlinks in which the target artifacts are listed as links to their respective artifacts. Fig. 6(b) details the corresponding XML trace link instance.

D. Automated Processes

Population of the traceability repository can be directly from generated dataset artifacts or through manual entry. This is implemented through the *traceability creation* process for datasets and the *manual trace generation* process for manual entry. The *insert repository* process inserts the traceability information in the traceability repository. The *traceability creation* process facilitates the insertion of the dataset traceability information in the traceability repository. First, the process reads all artifact instance files and calls the *insert repository* process to create a node with the corresponding properties for each artifact instance. Next, the process sequentially reads the trace links and traces link type instance files and calls the *insert repository* process to create the relations.

The *manual trace generation* process is called by the developer to link two artifacts. The developer inputs the properties for the source and target artifacts. The type of relation is generated from the trace type rule instance. Then the process calls the *insert repository* process to create the nodes and relations within the traceability repository. To view and manipulate the traceability information within the repository, we currently use the Neo4j browser and Cypher graph query language.

III. COMPARISONS AND RESULTS

This section presents a comparison between our implementation of the Neo4j graph database as a traceability repository and the current methods used for storage and representation of trace links. We begin by describing the datasets used in our experiments followed by the results of the comparisons.

A. Comparison Datasets

To provide a comparison between different methods we select datasets that represent real world artifacts and their dependencies. Each dataset is composed of a set of artifacts, their properties, and the relationships between the artifacts. All datasets do not provide semantic meanings of the trace links between artifacts. Therefore, we appended each dataset with trace type rules based on our understanding of the relationships between the different artifacts. Given that the preparation phase of our methodology is currently manual, for each dataset, we selected the most representative artifacts and trace links to allow the full application of our methodology and to allow for a complete evaluation and comparison between the different representation methods.

The first dataset, the *Event Based Traceability for Managing Evolutionary Change* (EBT), is a public dataset provided by CoEST [24]. EBT is a small dataset with artifacts represented in XML and text format. From the EBT dataset we selected the *requirements* and *test case*

artifacts for our evaluations. EBT provides one-to-one relations between artifacts. Table I details the EBT artifacts, its trace links and our trace link types.

The second dataset represents an industrial dataset for the customization of OpenERP [25], an open source ERP. The dataset was collected from a commercial application lifecycle management tool used by developers applying an agile methodology. The trace links within the dataset were entered manually by the project development team. The output from the tool was an Excel file that lists the contents of the artifacts. OpenERP is a medium size dataset in which we selected the *user story*, *task* and *developer test* artifacts as shown in Table I. The dataset includes one-to-one trace links between artifacts.

The third dataset is the Aqualush irrigation system used as a case study in [26]. We used the artifacts and links provided in [23], in which all artifacts and trace links are in HTML format. As Aqualush is a large dataset, for our work, we selected the *software requirements specification*, *user level requirements*, *use case*, *detailed design* and *software architecture* artifacts from the complete Aqualush dataset. In contrast to the previous datasets, Aqualush provides many-to-many trace links.

B. Visualization of Trace Links

Trace link visualization improves the ability of the developer to rapidly comprehend relationships between different artifacts. In this Section, we compare the visualization capability of traceability matrices, cross reference tables and graphs. We do not compare in terms of one-to-one relationships as the visualizations are similar for all types of representations [3].

1) *One-to-many relations*: in which one artifact is linked with more than one other artifact. Fig. 7(a) illustrates a one-to-many relation from the Aqualush dataset. For example, the *software requirements specification*: node SRS7 in the center has two *require* trace links with the *use cases*: nodes UC16, UC73 and 9 *fulfill* trace links with the 9 nodes representing the *software architecture* artifacts and two *satisfy* trace links with the *user level requirements*: nodes ULR59, ULR65. From Fig. 7(b), the cross reference representation visualizes the same relations by listing all linked (target) artifacts opposite the source artifact. In Fig. 7(c) three traceability matrices are used to represent the same relationships, as each matrix can only represent two dimensional relations.

2) *Many-to-many relations*: the representation of many-to-many relations is usually represented as a combination of two or more one-to-many relations. Fig. 8 illustrates the OpenERP dataset in which the three tasks: TS-22721, TS-30615, TS-30589 have many relations with different artifacts. The graph representation in Fig. 8 clearly visualizes these relationships. The cross reference representation (not shown in the Fig. 8) of the same relations would be tabulated such that each source task has the related target artifacts only. A traceability matrix would require two matrices to represent the many-to-many relations, one for *developer test* against *task* and another for *task* against *user story*.

TABLE I. EXPERIMENTAL DATASETS: ARTIFACTS, TRACE LINKS AND TRACE TYPES

Dataset	Artifacts		Trace Links			Trace Types	
	Type	Num	From	To	Num	Type	Num
EBT	Requirement	41	Requirement	Test case	51	accept	51
	Test Case	25					
Open ERP	User Story	350	Task	User Story	1304	contain	1304
	Task	1323	Developer Test	Task	65	validate	65
	Developer Test	198					
Aqualush [23]	Software Requirements Specification (SRS)	396	SRS	UC SArch DD ULR	4038	contain	2637
	User Level Requirements (ULR)	48	DD	SRS SArch	1719	satisfy	382
	Use Case (UC)	74				require	205
	Detailed Design (DD)	85				fulfill	874
	Software Architecture (SArch)	15				define	32

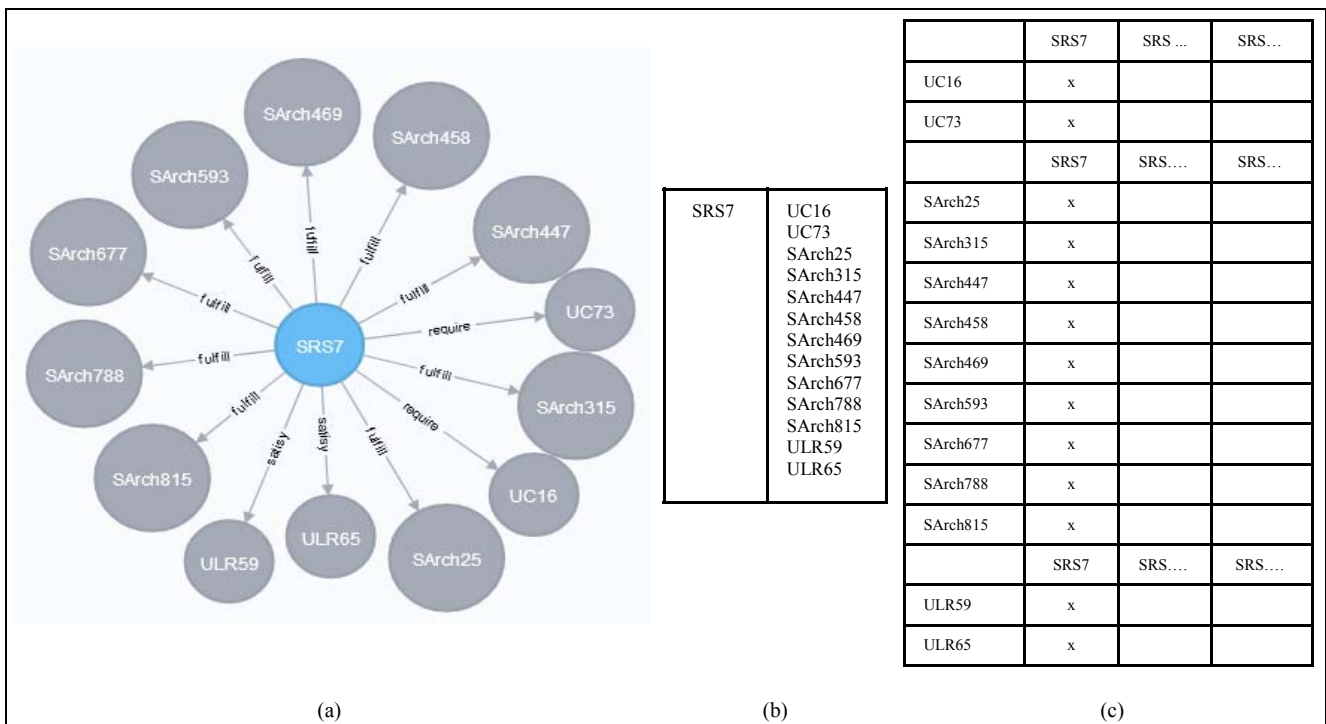


Fig. 7. One-to-many relation visualization: (a) graph, (b) cross reference and (c) traceability matrix.

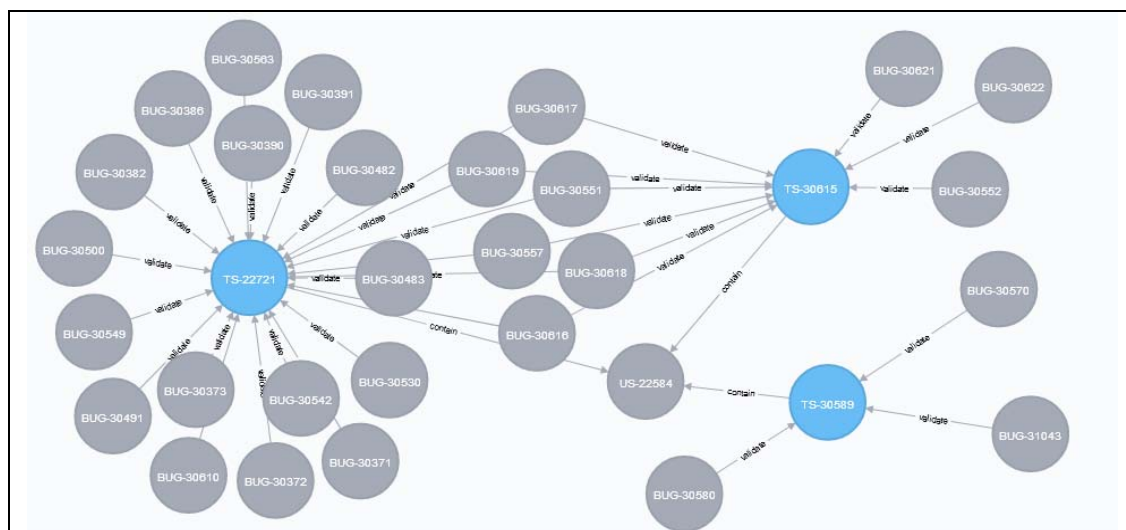


Fig. 8. Many-to-many relations: graph visualization.

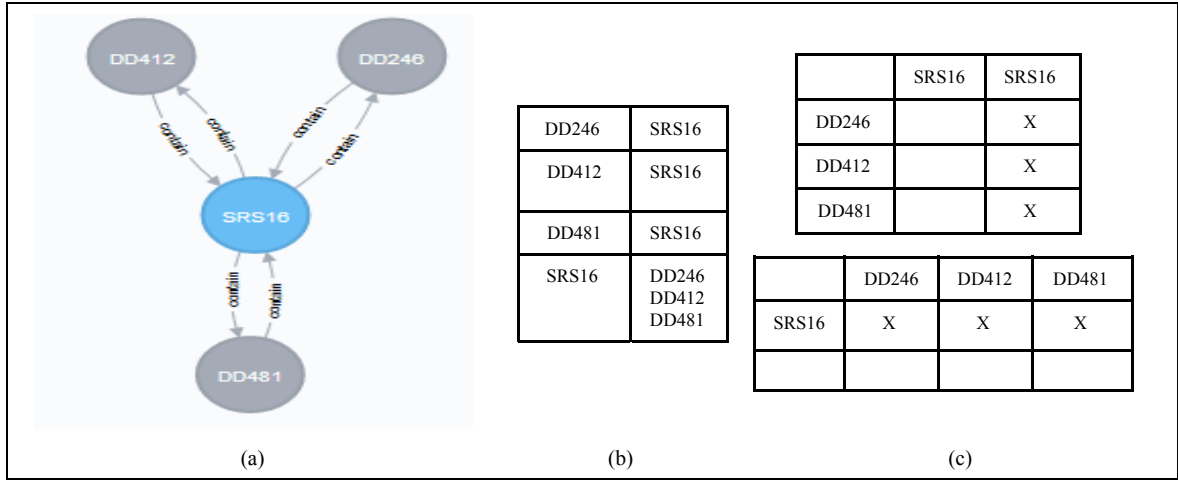


Fig. 9. Multi-directional relation visualization: (a) graph, (b) cross reference and (c) traceability matrix.

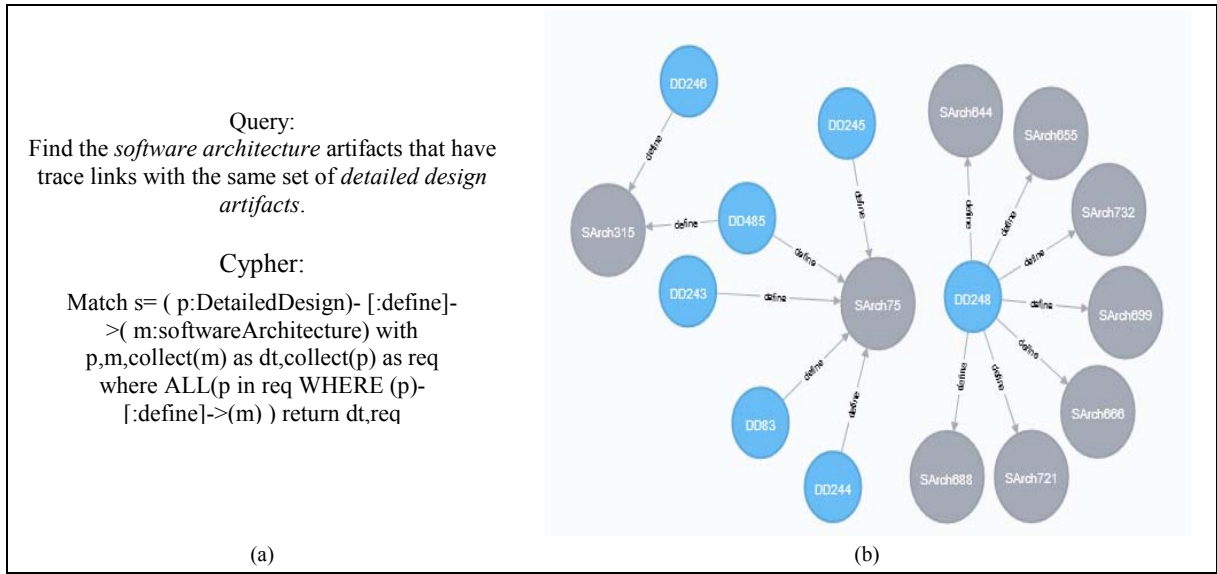


Fig. 10. The result set of a trace link derivation from the Aqualush dataset: (a) the query and (b) the derived trace links.

3) *Multi-direction trace links*: are trace links that relate the artifact to its source (backward direction) and to its derived artifacts (forward direction). Multi-direction trace links are expressed by an arrow at the end of the edge in the graph model. This is not the case in a traceability matrix which requires two cells to represent each direction as two relations, and cross reference tables require another row to represent the opposite direction relationship. Fig. 9 details a subset of the Aqualush dataset representing multi-directional links. In the graph visualization this is clearly represented by the direction of the edge between the nodes as can be seen in Fig. 9(a). In contrast, cross reference tables require four sets of linked artifacts, shown in Fig. 9(b), and the traceability matrix of Fig. 9(c) requires two traceability matrices.

C. Derivation of New Trace Links

Derivation of trace links is the generation or inferring of new trace links based on existing links. The implementation of a graph traceability repository allows for trace link inference through directly querying the repository for the existence of such relations. Fig. 10 provides such an example.

Using the Aqualush traceability repository, we write a query in Cypher to find the *software architecture* artifacts that have trace links with the same set of *detailed design* artifacts. Such a query can be used by a developer to assess impact change, e.g., *what are the detailed design artifacts that will need to be modified if the software architecture changes*. Fig. 10 details the output of the query: two sets, one in which 7 *software architecture* artifacts have a *define* relation with the *detailed design* node: DD248 and another in which *software architecture* artifact nodes: SArch75 and SArch315 share a relation with the *detailed design* artifact: DD485. Thus the developer, for example, may choose to update the traceability repository by linking the *detailed design* artifacts of SArch75 with those of SArch315.

The previous derivation is an example of semi-automated trace link generation, as the developer must manually link the artifacts based on the new derived links. Using Cypher, different relationships between artifacts can be investigated and new trace links added to the repository. This feature can be a basis to develop a metamodel for the semi-automated generation of trace

link derivation rules based on the current trace links within the repository.

Another aspect is the visualization of the returned result set. As can be seen from Fig. 10, the output of the query directly links the artifacts and displays the trace link types. This capability enhances the developer's decision making process and accurately represents the relationships between the artifacts. In contrast, the result set of an SQL query is a table of rows which will require an additional step to visualize the relationships between artifacts.

D. Performance in Terms of Query Response Times

In this Section, we investigate the performance of the Neo4j traceability repository in comparison to the structured file format implemented in most commercial lifecycle tools. To represent the traceability storage format of these tools, we used the structure of the MySQL database tables of the commercial tool used by the developers of the OpenERP dataset. The MySQL traceability information is stored in three tables: one for artifact properties, another for artifact linkage and a third for trace link types. All projects (datasets) are stored in these tables with fields to differentiate projects by name. To conduct the experiments, we used a MS Windows 7, core i7 CPU, 4 GB RAM laptop with Neo4j community edition 3.2.3 and MySQL 5.6 databases installed.

The Neo4j traceability repository contains the three datasets stated in the previous section, stored within three instances, each representing a database for a different dataset. The artifacts and trace links are created by our methodology, as discussed in Section II. For MySQL, the three datasets are stored within the same database based on the design of the commercial tool mentioned above. In addition, the dataset artifacts were created through the tool interface and the trace links were imported into the database directly from our datasets.

We wrote two queries: one pair in Cypher for the Neo4j database and the other pair in SQL for MySQL. Queries are specific to the artifacts and trace links for each dataset. Each query was executed three times and the average response times were reported. To verify that the queries were logically identical we compared the result sets for both databases. The first query (Q1) selects artifacts based on their trace type links with other artifacts. For example for the OpenERP dataset, Q1 selects all *task* artifacts that have a *contain* relation with *user story* artifacts and at the same time have a *validate* relation with *developer test* artifacts. For Cypher, Q1 requires filtering based on relation type only. For SQL, Q1 requires multiple joins in order to link the source artifact with the target artifact based on the trace link type.

The second query (Q2) finds all artifacts that have more than 20 relations with another artifact. An example for Q2 for the OpenERP dataset is shown in Fig. 11. Q2 finds all *user stories* that have more than 20 *contain* relations with *task* artifacts. Q2 includes a condition using the aggregate function *count*. The aggregate function is applied to filter the initial result set. This impacts the

performance of the database depending on the initial size of the non-filtered set.

Table II summarizes the mean query response times for Q1 and Q2 queries for all datasets. The response time for the Neo4j database is negligible for all queries and datasets in comparison to the performance of the MySQL queries. The impact of the dataset size and the number of trace links on the performance of the MySQL queries is evident: as the number of links increase, the join size increases and accordingly the performance of the query decreases. An interesting observation is the response times for the OpenERP dataset in which Q1 out performs Q2. This is due to the number of *contain* relations within the OpenERP dataset that represent an estimated 95% of all trace links. It is evident that the relational table structure is not suitable for traceability queries investigating the relationships between artifacts and trace links especially for large datasets.

Neo4j				
Match s= (m:Task)-[:contain]->(p:UserStories) with p, count(DISTINCT m) as cnt where cnt > 20 return p				
MySQL				
SELECT to_id, count(*), k.details FROM assoc a LEFT JOIN task k on k.summary=a.to_id and a.proj_id=2 WHERE (proj_id=2 and assoc_id=3) GROUP BY to_id HAVING count(*) > 20;				

Fig. 11. The Q2 query for the OpenERP dataset.

TABLE II. QUERY RESPONSE TIMES FOR Q1 AND Q2 ON NEO4J AND MYSQL DATABASES

EBT				
<i>Query</i>	<i>Neo4j</i>		<i>MySQL</i>	
	<i>Result Set</i>	<i>Response time (seconds)</i>	<i>Result Set</i>	<i>Response time (seconds)</i>
Q1	64 node 51 relations	>0.001	51 records	0.29
Q2	15 nodes	>0.001	15 records	0.15
OpenERP				
Q1	78 nodes 80 relations	>0.001	80 records	0.52
Q2	11 nodes	>0.001	11 records	2.9
Aqualush				
Q1	371 nodes 716 relations	0.11	716 records	4.52
Q2	12 nodes	>0.001	12 records	2.14

E. Lessons Learned

We have applied our methodology to different datasets, with varying numbers of artifacts and trace links. The metamodel patterns were easily modified and reapplied for each dataset. The metamodel instances provide the structure of the artifacts and trace links which form the base of the Cypher queries executed by the processes. This provided data validation and error correction for inconsistencies due to the manual preparation of the artifact and traceability files. In addition, based on our experiments, the graph database performance is suitable for an industrial setting. For example, creating the Aqualush trace links, which exceed 4000 links, took less than two minutes. Similarly, the removal of the links completed after 68 ms. We would expect better response times on faster machines. An important factor was the ease of use of the Cypher query language and its ability to query the repository in straight forward syntax in comparison to SQL. Cypher has similar data manipulation capabilities as other query languages, therefore special-purpose traceability representation and querying languages were not required.

The Neo4j visualization capabilities naturally represent trace relations and thus are expected to increase developer understanding and overall productivity. We anticipate that incorporating similar graph traceability repositories within lifecycle tools will enhance the practice of traceability within software development. Our current methodology has been implemented as a semi-automated process. The automation of the metamodel patterns and the dataset preparation phase is straightforward.

IV. CONCLUSION AND FUTURE WORK

Traceability identifies dependencies between software artifacts allowing developers to analyze the impact of modifications to requirements, design and code throughout the development lifecycle. There is limited application of traceability in industry due to the complexity of traceability models and lack of tool implementation. This paper has introduced a user-defined traceability metamodel that consists of XML patterns for defining artifacts, trace links and trace link type rules. Our metamodel patterns are simpler than current approaches in the literature. In addition, the trace link type rule patterns allow for a user-defined semantic classification of the dependencies between artifacts. We presented graph traceability repositories as an alternative form for representing and storing traceability information. We demonstrated the advantage of graph repositories in comparison to traceability matrices, cross reference tables and relational databases in terms of visualization of trace relations, representation of multi-dimensional relations and query response times.

For future work, we will automate all steps of our methodology and implement trace link derivation through the semantic relations described within the trace link types. To ensure applicability and usability we will implement our methodology and graph traceability repository as a component within an open source lifecycle tool. We aim to provide a base for developing more

flexible traceability tools that will enhance software reliability and quality during the software development lifecycle.

REFERENCES

- [1] O. C. Gotel and C. Finkelstein, "An analysis of the requirements traceability problem," *Requirements Engineering '94*, pp. 94-101.
- [2] O. Gotel, J. Cleland-Huang, J. H. Hayes, A. Zisman, A. Egyed, P. Grünbacher, et al., "Traceability fundamentals," *Software and Systems Traceability*, 2012, pp. 3-22.
- [3] S. Winkler and J. Pilgrim, "A survey of traceability in requirements engineering and model-driven development," *SoSyM*, vol. 9, pp. 529-565, 2010.
- [4] G. Spanoudakis and A. Zisman, "Software traceability: a roadmap," *Handbook of Software Engineering and Knowledge Engineering*, vol. 3, pp. 395-428, 2005.
- [5] J. Cleland-Huang, C. K. Chang, and Y. Ge, "Supporting event based traceability through high-level recognition of change events", *COMPSAC 2002*, pp. 595-600.
- [6] B. Ramesh and M. Jarke, "Toward reference models for requirements traceability," *IEEE TSE*, vol. 27, pp. 58-93, 2001.
- [7] A. Espinoza, P. P. Alarcon, and J. Garbajosa, "Analyzing and systematizing current traceability schemas," *30th Annual IEEE/NASA Software Engineering Workshop 2006*, pp. 21-32.
- [8] P. Letelier, "A framework for requirements traceability in UML-based projects," *TEFSE 2002*, pp. 173-183.
- [9] D. Bildhauer, J. Ebert, V. Riediger, and H. Schwarz, "Using the TGraph approach for model fact repositories," *MoRSe*, 2008.
- [10] H. Schwarz, J. Ebert, and A. Winter, "Graph-based traceability: a comprehensive approach," *SoSyM*, vol. 9, pp. 473-492, 2010.
- [11] B. Vanhooff, S. Van Baelen, W. Joosen, and Y. Berbers, "Traceability as input for model transformations," *ECMDA-TW 2007*, pp. 37-46.
- [12] A. Espinoza and J. Garbajosa, "A study to support agile methods more effectively through traceability," *Innovations in Systems and Software Engineering*, vol. 7, pp. 53-69, 2011.
- [13] G. Spanoudakis, A. Zisman, E. Pérez-Minana, and P. Krause, "Rule-based generation of requirements traceability relations," *JSS*, vol. 72, pp. 105-127, 2004.
- [14] J. Cleland-Huang, B. Berenbach, S. Clark, R. Settini, and E. Romanova, "Best practices for automated traceability," *Computer*, vol. 40, 2007.
- [15] M. Lormans and A. Van Deursen, "Reconstructing requirements coverage views from design and test using traceability recovery via LSI," in *TEFSE 2005*, pp. 37-42.
- [16] F. Erata, M. Challenger, B. Tekinerdogan, A. Monceaux, E. Tüzün, and G. Kardas, "Tarski: a platform for automated analysis of dynamically configurable traceability semantics," *Symposium on Applied Computing*, 2017, pp. 1607-1614.
- [17] A. Goknil, I. Kurtev, and K. Van Den Berg, "Generation and validation of traces between requirements and architecture based on formal trace semantics," *JSS*, vol. 88, pp. 112-137, 2014.
- [18] R. Elamin and R. Osman, "Towards Requirements Reuse by Implementing Traceability in Agile Development," *COMPSAC 2017*, pp. 431-436.
- [19] J. H. Hayes, A. Dekhtyar, and J. Osborne, "Improving requirements tracing via information retrieval," in *11th Requirements Engineering Conference*, 2003, pp. 138-147.
- [20] P. Heim, S. Lohmann, K. Lauenroth, and J. Ziegler, "Graph-based visualization of requirements relationships," *REV 2008*, pp. 51-55.
- [21] M. Rath, D. Akehurst, C. Borowski, and P. Mäder, "Are Graph Query Languages Applicable for Requirements Traceability Analysis?," *REFSQ 2017*.
- [22] S. Winkler, "On usability in requirements trace visualizations," in *REV 2008*, pp. 56-60.
- [23] E. Ben Charrada, D. Caspar, C. Jeanneret, and M. Glinz, "Towards a benchmark for traceability," in *Joint EVOL and IWPSE 2011*, pp. 21-30.
- [24] <http://www.coest.org>
- [25] <https://www.odoo.com>
- [26] C. Fox, *Introduction to Software Engineering Design: Processes, Principles and Patterns with UML2*. Addison-Wesley, 2006.